# Mini-Project 2

CPE333 Operating Systems

# Critical Section

- Group of around 6-8 students
- Study on an assigned topic
- Re-build kernel files to achieve assigned tasks
- Make an ultra-presentation within 7-11 Minutes
  - Explain the studying topic (fancy present is allowed)
  - Demo your implementation
- Make a report with no more than 25 pages. (excluding appendix)

# Topics

- Process Scheduler Profiler
  - Observe how Linux actually schedules and migrates processes
- Virtual Memory
  - Explore VMAs, page faults, residency, and demand paging.
- I/O and File system
  - Implement a simplified filesystem through the Linux VFS.

# Topic: CPU Virtualization

- Create Process Scheduler Profiler
  - Get input as process_id that you want to monitor (via /proc or command line arguments)
  - Monitor scheduling activities of that process.
  - Show how many times the process is scheduled in and scheduled out.
  - Show how many times the process is woken up.
  - Show how many times the process moves between CPU cores.

- Hint
  - look at ftrace and perf
  - for kernel module look at tracingpoint sched_wakeup, sched_switch, sched_migrate_task here: https://github.com/torvalds/linux/blob/master/kernel/tracepoint.c
  - and for further explaination can be found here: https://docs.kernel.org/trace/tracepoints.html
  - look further into DECLARE_TRACE vs TRACE_EVENT

# Topic: Virtual Memory

- Create a page fault counter tool for a specific process
  - Get input as process_id that you want to monitor
  - Show a summary containing:
    - Number of memory regions
    - Total virtual address space
    - Minor page faults
    - Major page faults
- Hint
  - for kernel module look at tracingpoint https://github.com/torvalds/linux/blob/master/include/trace/events/exceptions.h
  - and for further explaination can be found here: https://docs.kernel.org/trace/tracepoints.html
  - A process's address space is divided into Virtual Memory Areas (VMAs). Investigate struct mm_struct and struct vm_area_struct and get_task_mm function then count the VMAs to obtain the number of memory regions, and calculate each region size from its start and end addresses. Compare your result with /proc/<pid>/maps.
  - More info on process’s address: https://docs.kernel.org/mm/process_addrs.html

# Topic: I/O and File System

- Mini RAM-Based File System
  - Implement the RAM-Based filesystem.
  - Allow the filesystem to be mounted to a directory.
  - Support basic filesystem operations:
    - Mount
    - Unmount
    - Create file
    - Read file
    - Write file
    - Create directory
    - List directory contents
- Hint:
  - How VFS work in linux: https://docs.kernel.org/filesystems/vfs.html
  - Study from ramfs: https://github.com/torvalds/linux/blob/master/fs/ramfs/inode.c